Question 1: Different Generative Models

This section is composed of **20 single-choice questions** divided into four distinct sub-parts, with each part carrying equal marks:

- **Text Generation:** 5 questions covering tokenization, teacher forcing, and KV caching.
- **Audio Generation:** 5 questions regarding waveforms, spectrograms, voice cloning, and quantization.
- **3D Generation:** 5 questions on Signed Distance Functions (SDF), NeRFs, Gaussian Splatting, and Score Distillation Sampling (SDS).
- **Video Generation:** 5 questions focused on optical flow, temporal consistency, and latent space operations.

Question 2: Basics of Generative Models

This section focuses on the foundational architectures and mathematical mechanics of deep learning and generative modelling. The marking for this question is weighted as follows:

- **Softmax Classifier (15%):** Requires calculating cross-entropy loss and gradients.
- **Convolutional Neural Networks (20%):** Involves calculating output dimensions, trainable parameters, and receptive fields.
- **Transformers and Attention (55%):** The largest portion, covering scaled dot-product attention, complexity, BERT vs. GPT objectives, and Vision Transformers (ViT).
- **Low-Rank Adaptation (LoRA) (10%):** Focuses on the rank and parameter efficiency of LoRA adapters.

Question 3: VAE, GAN, and Diffusion

This section evaluates high-level generative frameworks through a mix of mathematical derivations and conceptual questions. The marking is weighted as follows:

- **KL Divergence (30%):** Definitions, symmetry, and Gaussian-specific calculations.
- **Variational Autoencoders (20%):** Deriving the Evidence Lower Bound (ELBO) and the reparameterization trick.
- **Generative Adversarial Networks (20%):** Minimax objectives and the roles of the generator and discriminator.
- **Diffusion Models (30%):** Forward process Markov chains, T-step closed-form expressions, and the "Generative Trilemma".

Time Allocation Strategy

The total marks are distributed across three questions. Based on the document, Question 1 consists of 20 rapid-fire single-choice questions, while Questions 2 and 3 require derivations and calculations.

Section	Format	Recommended Time	Goal
Question 1	20 Single-Choice Questions	20 Minutes	1 minute per question.
Question 2	Technical/Math (CNNs, Transformers)	30 Minutes	Detailed calculations.
Question 3	Theory/Derivations (VAE, GAN, Diffusion)	40 Minutes	Multi-step proofs.

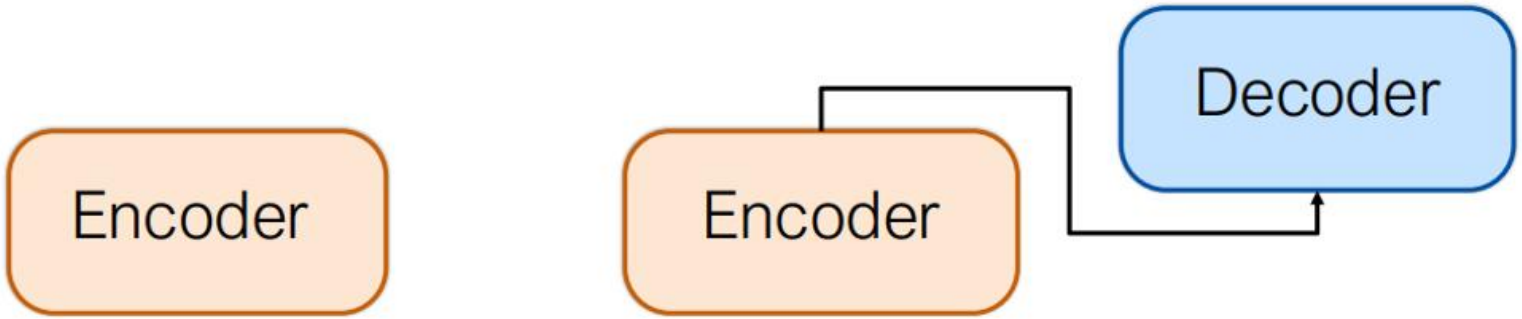
- The instructions explicitly state **not to evaluate logarithms numerically**.
- **Avoid Sunk Cost:** Spend no more than 2 minutes attempting a problem without a clear path forward. Ensure you **leave space** after skipped items to maintain a clean layout for second-pass attempts.

Type of Language Models

Encoder-only
Masked LM
Non-autoregressive

Encoder-decoder

Decoder-only
Autoregressive



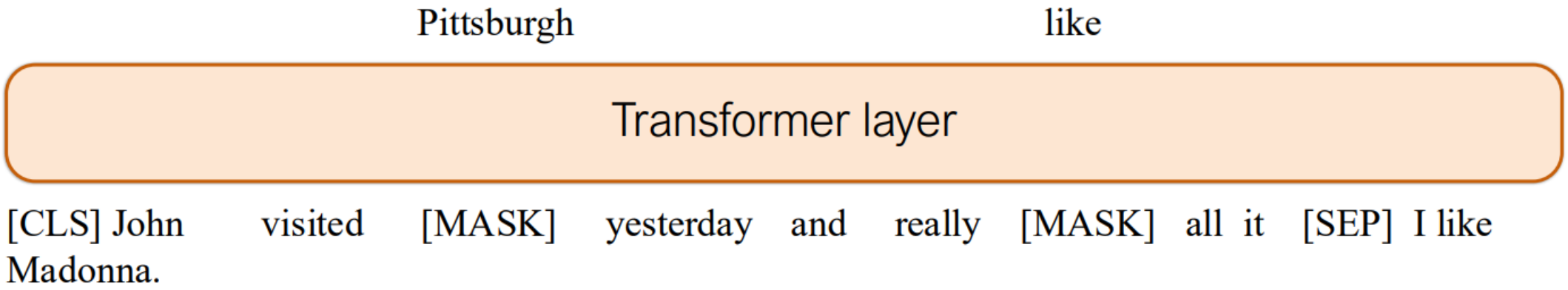
e.g. BERT
RoBERTa
ESM (for protein)

e.g. T5

e.g. GPT
LLaMA
ProGen (for protein)

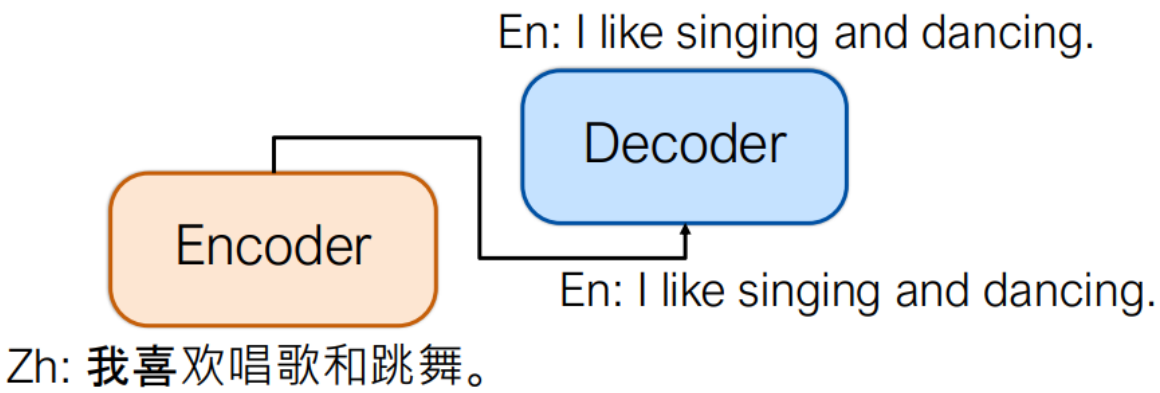
Encoder-only Language Model

- Masked Prediction: BERT $P(x_{mask}|x)$
- Can also be used to generate language



Encoder-Decoder Language Model

- The model learns the conditional probability $P(Y|X)$ of a target sequence given a source sequence.
- **Encoder:** Processes the input sequence. Common implementations include Bi-directional LSTMs, Transformers, or CNNs.
- **Decoder:** Generates the output sequence. Implementations include LSTMs, Transformers, or Non-autoregressive Transformers.

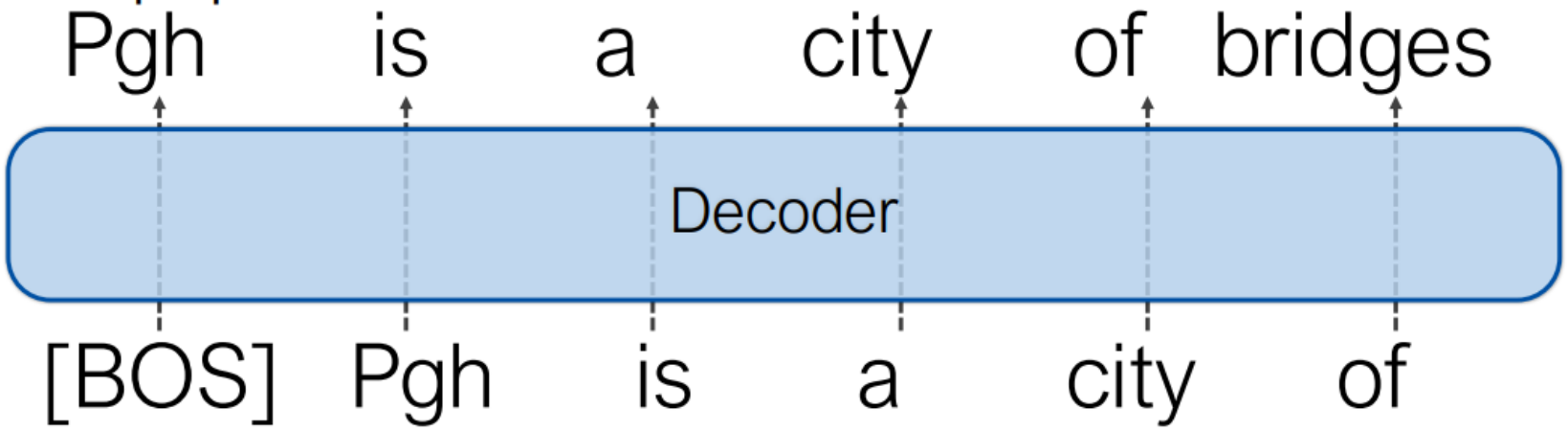


T5 (Text-to-Text Transfer Transformer) — *Google*

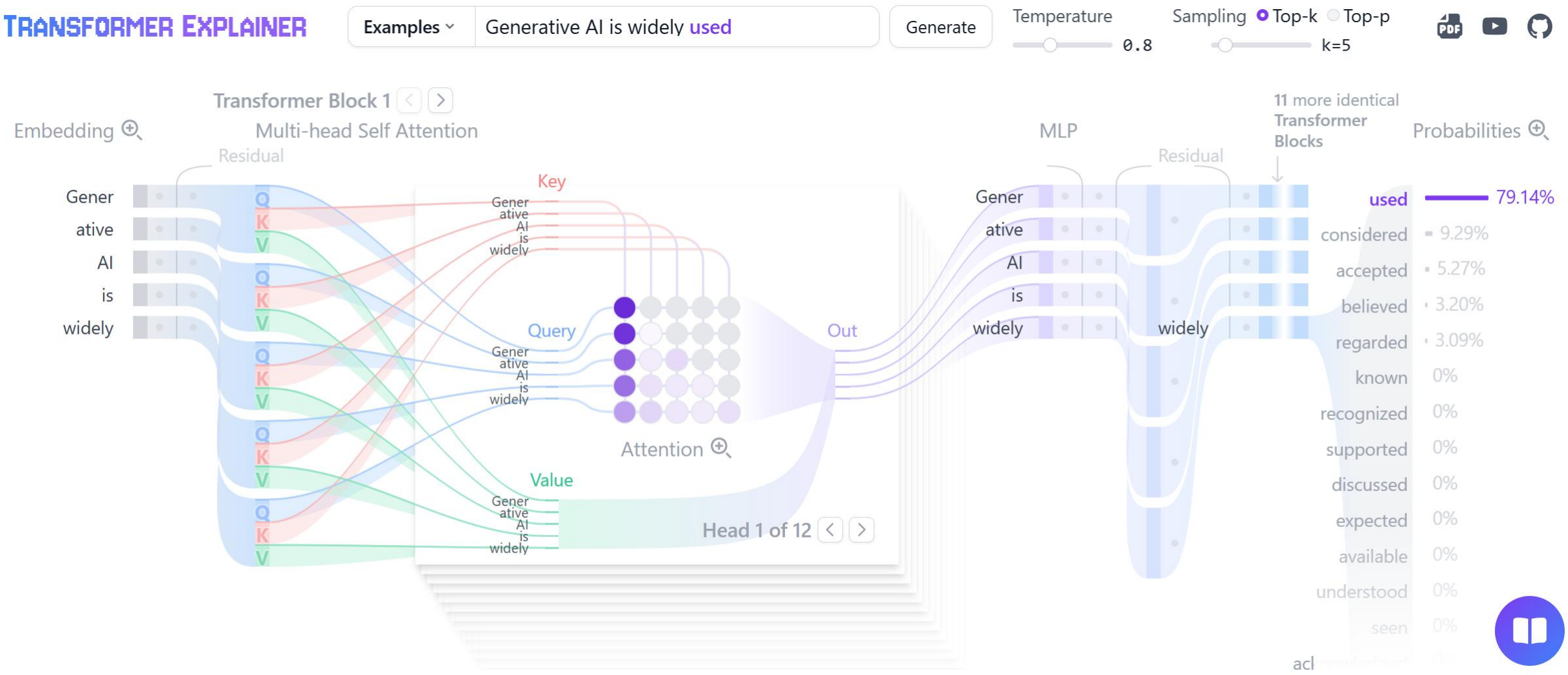
T5 is likely the most famous modern implementation of the pure Encoder-Decoder architecture.

Decoder-only Language Model

- Causal Language Model: $P(X) = \prod_{n=1}^{N} P(x_n | x_{1..n-1})$
- Model choice: Transformer or (uni-directional) LSTM
- Most popular choice of LLM architecture



GPT-2 （small）



The GPT-2 (small) model has 124 million parameters (Transformer Explainer).

Tokenizer – Split Text into Basic Units

Many words don't map to one token: indivisible.

↓ tokenizer

Many words don't map to one token: indivisible.

[7085, 2456, 836, 470, 3975, 284, 530, 11241, 25, 773, 452, 12843, 13]

↓ embedding table lookup

2.3	-3.2	8.3	5.4	2.1	3.9	-8.9	3.8	3.9	3.3
4.5	5.9	4.5	7.1	1.0	5.3	5.0	3.1	0.7	5.0
…	…	…	…	…	…	…	…	…	…
3.8	1.2	3.8	9.0	9.3	3.1	4.2	0.8	9.2	5.8

Word-level Tokenization

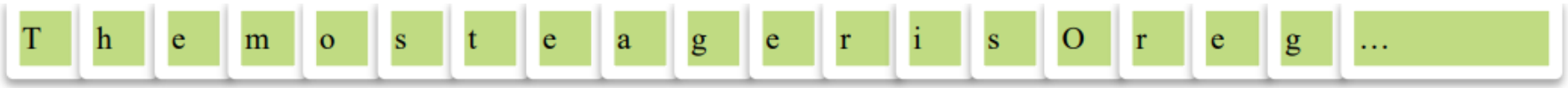
- Word-level Tokenization
  - Break by space and punctuation.
  - English, French, German, Spanish

Small Vocabulary	Large Vocabulary
✅ **Pros:** Fewer parameters (faster, lighter model).	✅ **Pros:** Fewer "Unknown" words (better coverage).
❌ **Cons:** High **OOV** (Out-Of-Vocabulary) rate.	❌ **Cons:** More parameters to learn (harder to train, slower generation).
(e.g., New words like "Covid" become `<UNK>` *)*	

The most eager is Oregon which is enlisting 5,000 drivers in the country's biggest experiment.

- Cons:
  - Out-of-vocabulary (OOV) or unknown tokens, e.g. Covid
  - Tradeoff between parameters size and unknown chances.
    - Smaller vocab => fewer parameters to learn, easier to generate (deciding one word from smaller dictionary), more OOV
    - Larger vocab => more parameters to learn, harder to generate, less OOV
  - Hard for certain languages with continuous script: Japanese, Chinese, Korean, Khmer, etc. Need separate word segmentation tool (can be neural networks)

Character-level Tokenization



- Each letter and punctuation is a token
- Pros:
  - Very small vocabulary (except for some languages, e.g. Chinese)
  - No Out-of-Vocabulary token
- Cons:
  - A sentence can be longer sequence
  - Tokens do not representing semantic meaning

Subword-level Tokenization

The most eager is Oregon which is en listing 5,000 driver s in the country 's big g est experiment.

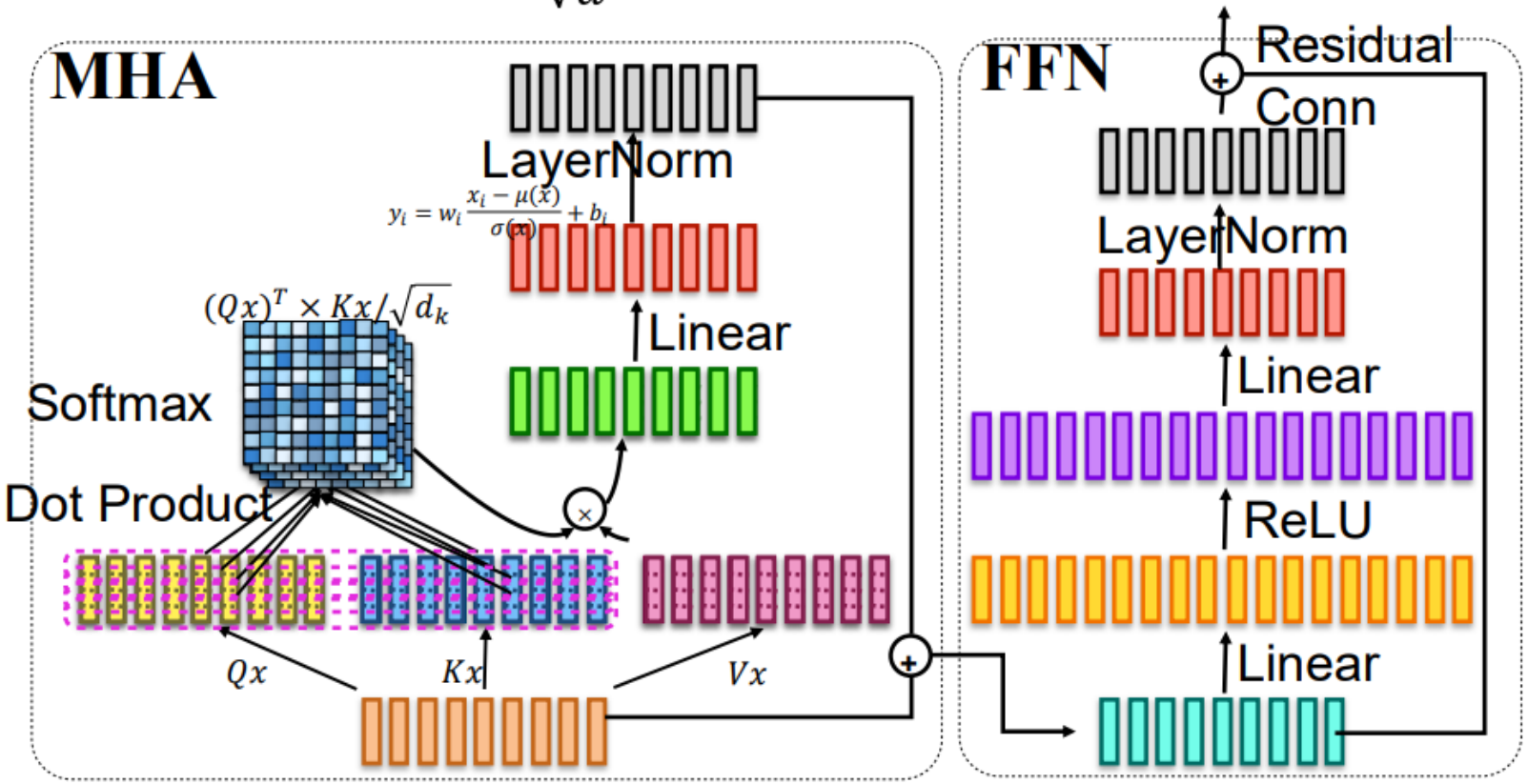
- Goal:
  - moderate size vocabulary
  - no OOV
- Idea:
  - represent rare words (OOV) by sequence of subwords
- Byte Pair Encoding (BPE)
  - not necessarily semantic meaningful
  - Originally for data compression Philip Gage. A New Algorithm for Data Compression, 1994

Online Demo:
https://koala.sh/tools/free-gpt-tokenizer
https://belladoreai.github.io/llama-tokenizer-js/example-demo/build/

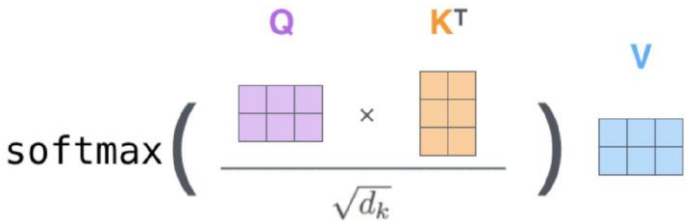
Multi-head Attention and FFN

$$\text{Attention}(Q,K,V,x) = \text{Softmax}\left(\frac{(Qx)^T Kx}{\sqrt{d}}\right) \cdot (Vx)^T$$

$$\text{FFN}(x) = max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$$



Attention Scaling



Softmax(| 10 | 4 | / | -5 | 2 |) = | 0.99 | 2e-3 | / | 9e-4 | 0.99 |

Softmax pushes disparity to extremes => vanishing gradients

2	1	3
1	-2	-1

⊗

1	3
2	1
2	-1

=

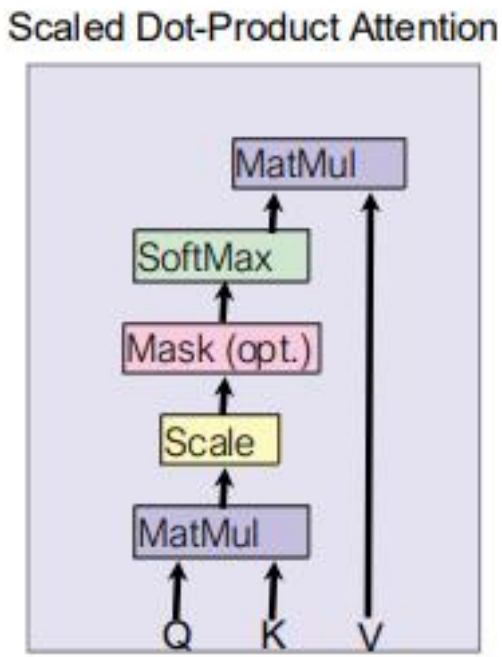
10	4
-5	2

Softmax(| 10 | 4 | / | -5 | 2 | / sqrt(3)) = | 0.97 | 0.03 | / | 0.02 | 0.98 |

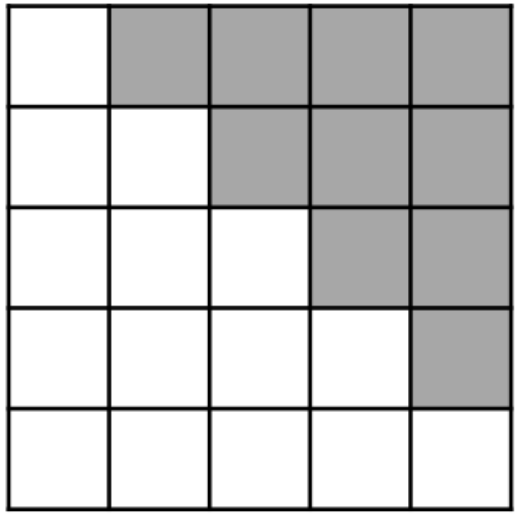
Scaling reduces the vanishing gradient!

https://jalammar.github.io/illustrated-transformer/

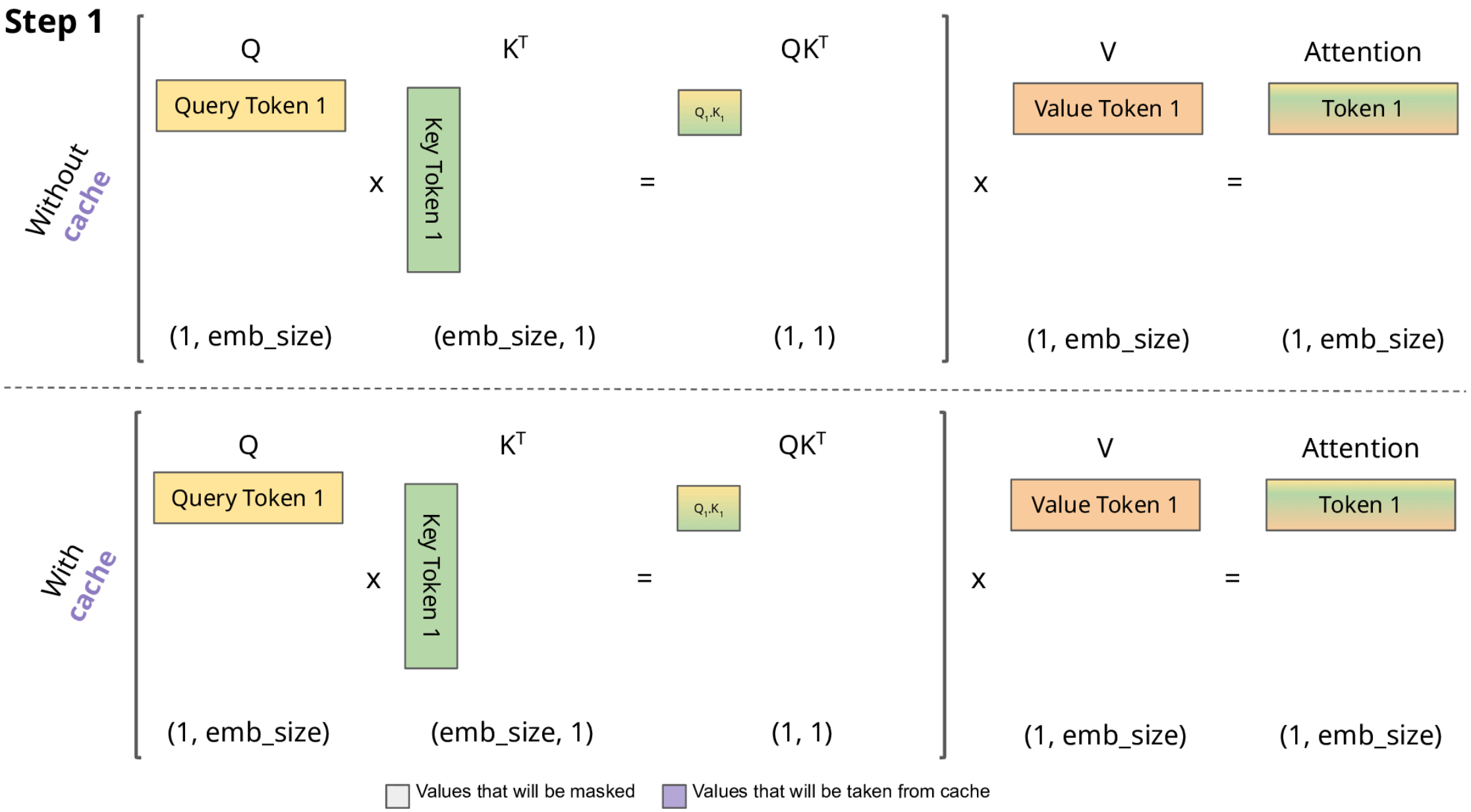
Causal Attention



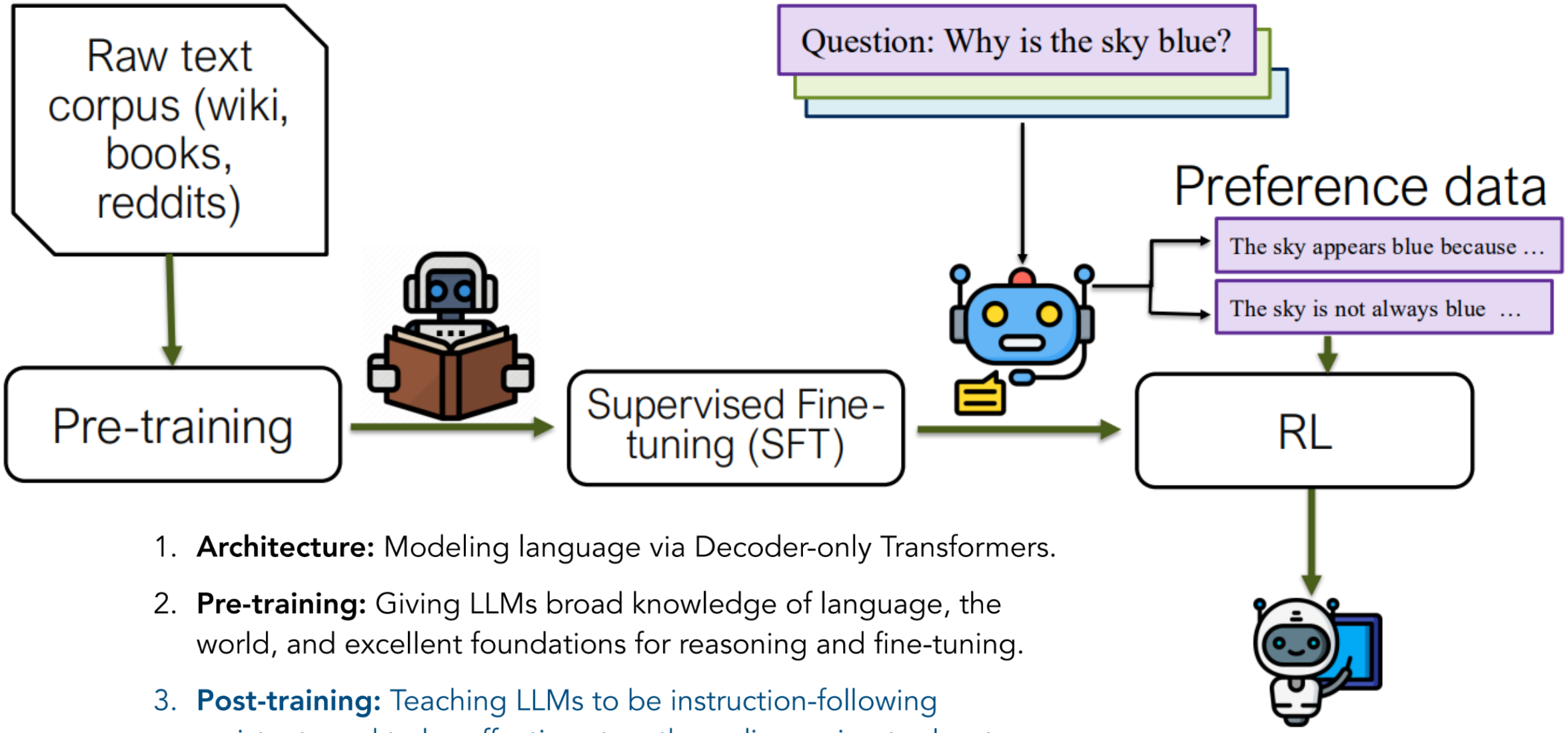
Maskout right side before softmax (-inf)



KV-Cache

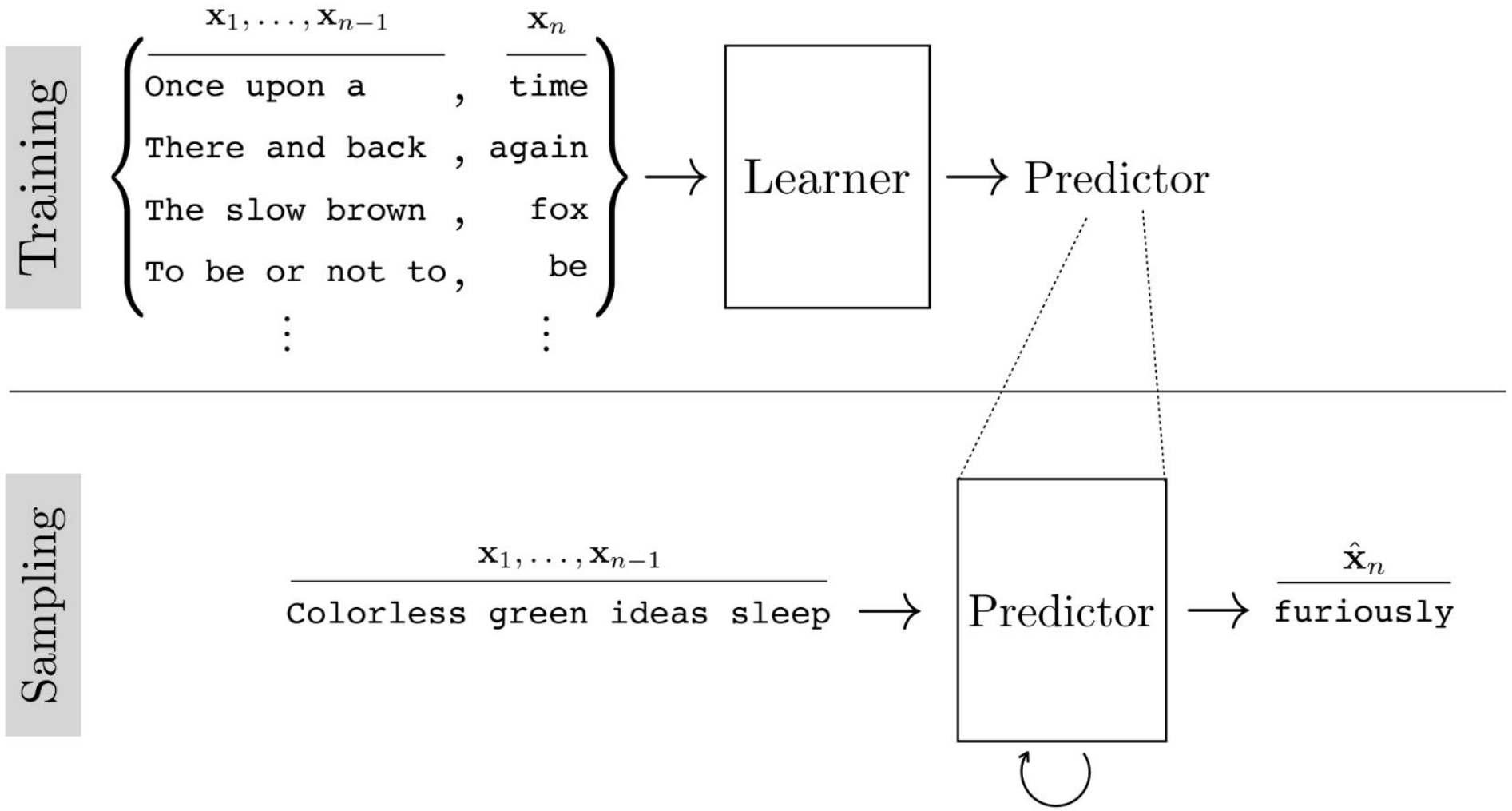

Step 1
Without cache
Q
Query Token 1
(1, emb_size)
x
K^T
Key Token 1
(emb_size, 1)
=
QK^T
Q1.K1
(1, 1)
x
V
Value Token 1
(1, emb_size)
=
Attention
Token 1
(1, emb_size)
With cache
Q
Query Token 1
(1, emb_size)
x
K^T
Key Token 1
(emb_size, 1)
=
QK^T
Q1.K1
(1, 1)
x
V
Value Token 1
(1, emb_size)
=
Attention
Token 1
(1, emb_size)
Values that will be masked
Values that will be taken from cache

LLM Learning Framework



1. **Architecture:** Modeling language via Decoder-only Transformers.
2. **Pre-training:** Giving LLMs broad knowledge of language, the world, and excellent foundations for reasoning and fine-tuning.
3. **Post-training:** Teaching LLMs to be instruction-following assistants and to be effective at math, coding, using tools, etc.

Autoregressive Models



Embedding

Step 1: Tokenization

Tokenization is the process of breaking down the input text into smaller, more manageable pieces called tokens. These tokens can be a word or a subword. The words “AI“, “is" and “widely" correspond to unique tokens, while the word “Generative" is split into two tokens. The full vocabulary of tokens is decided before training the model: GPT-2's vocabulary has **50,257 unique tokens**. Now that we split our input text into tokens with distinct IDs, we can obtain their vector representation from embeddings.

Step 2. Token Embedding

GPT-2 (small) represents each token in the vocabulary as a 768-dimensional vector; the dimension of the vector depends on the model. These embedding vectors are stored in a matrix of shape **(50,257, 768),** containing approximately **39 million parameters**! This extensive matrix allows the model to assign semantic meaning to each token, in the sense that tokens with **similar** usage or meaning in language are placed **close together** in this high-dimensional space, while **dissimilar** tokens are **farther apart**.

Tokenizer – Split Text into Basic Units

Many words don't map to one token: indivisible.

↓ tokenizer

Many words don't map to one token: indivisible.

[7085, 2456, 836, 470, 3975, 284, 530, 11241, 25, 773, 452, 12843, 13]

↓ embedding table lookup

2.3	-3.2	8.3	5.4	2.1	3.9	-8.9	3.8	3.9	3.3
4.5	5.9	4.5	7.1	1.0	5.3	5.0	3.1	0.7	5.0
…	…	…	…	…	…	…	…	…	…
3.8	1.2	3.8	9.0	9.3	3.1	4.2	0.8	9.2	5.8

Word-level Tokenization

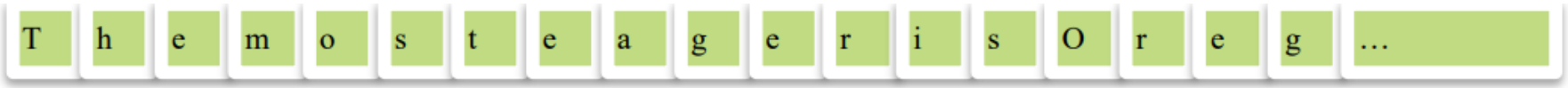
- Word-level Tokenization
  - Break by space and punctuation.
  - English, French, German, Spanish

Small Vocabulary	Large Vocabulary
✅ **Pros:** Fewer parameters (faster, lighter model).	✅ **Pros:** Fewer "Unknown" words (better coverage).
❌ **Cons:** High **OOV** (Out-Of-Vocabulary) rate.	❌ **Cons:** More parameters to learn (harder to train, slower generation).
(e.g., New words like "Covid" become `<UNK>` *)*	

The most eager is Oregon which is enlisting 5,000 drivers in the country's biggest experiment.

- Cons:
  - Out-of-vocabulary (OOV) or unknown tokens, e.g. Covid
  - Tradeoff between parameters size and unknown chances.
    - Smaller vocab => fewer parameters to learn, easier to generate (deciding one word from smaller dictionary), more OOV
    - Larger vocab => more parameters to learn, harder to generate, less OOV
  - Hard for certain languages with continuous script: Japanese, Chinese, Korean, Khmer, etc. Need separate word segmentation tool (can be neural networks)

Character-level Tokenization



- Each letter and punctuation is a token
- Pros:
  - Very small vocabulary (except for some languages, e.g. Chinese)
  - No Out-of-Vocabulary token
- Cons:
  - A sentence can be longer sequence
  - Tokens do not representing semantic meaning

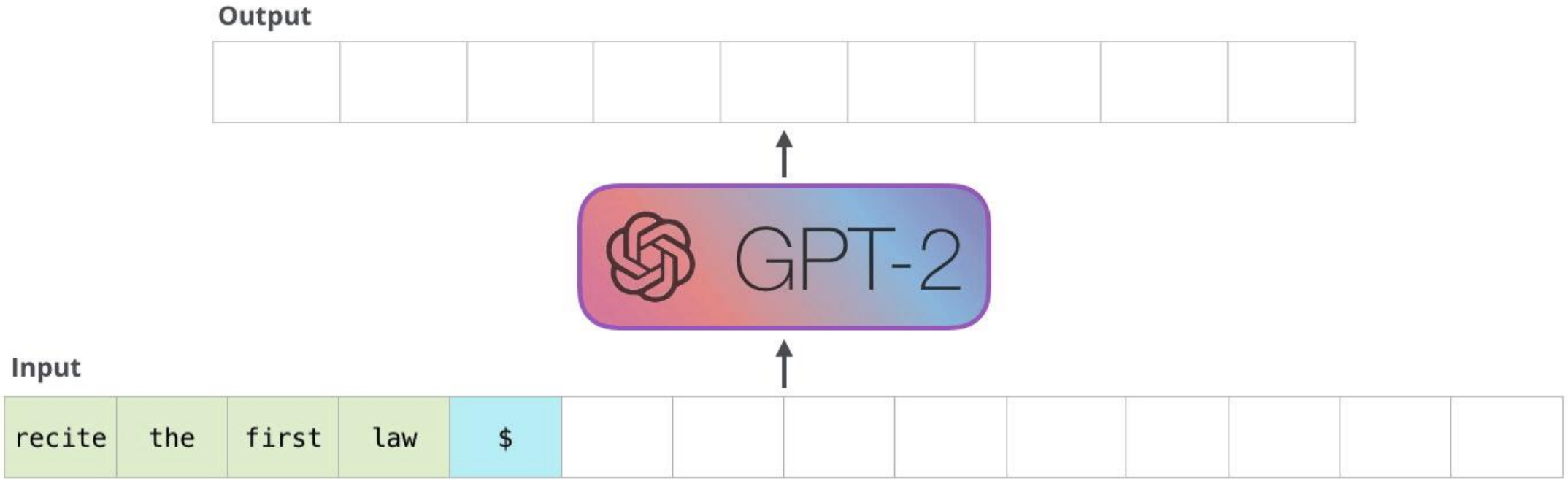
Subword-level Tokenization

The most eager is Oregon which is en listing 5,000 driver s in the country 's big g est experiment.

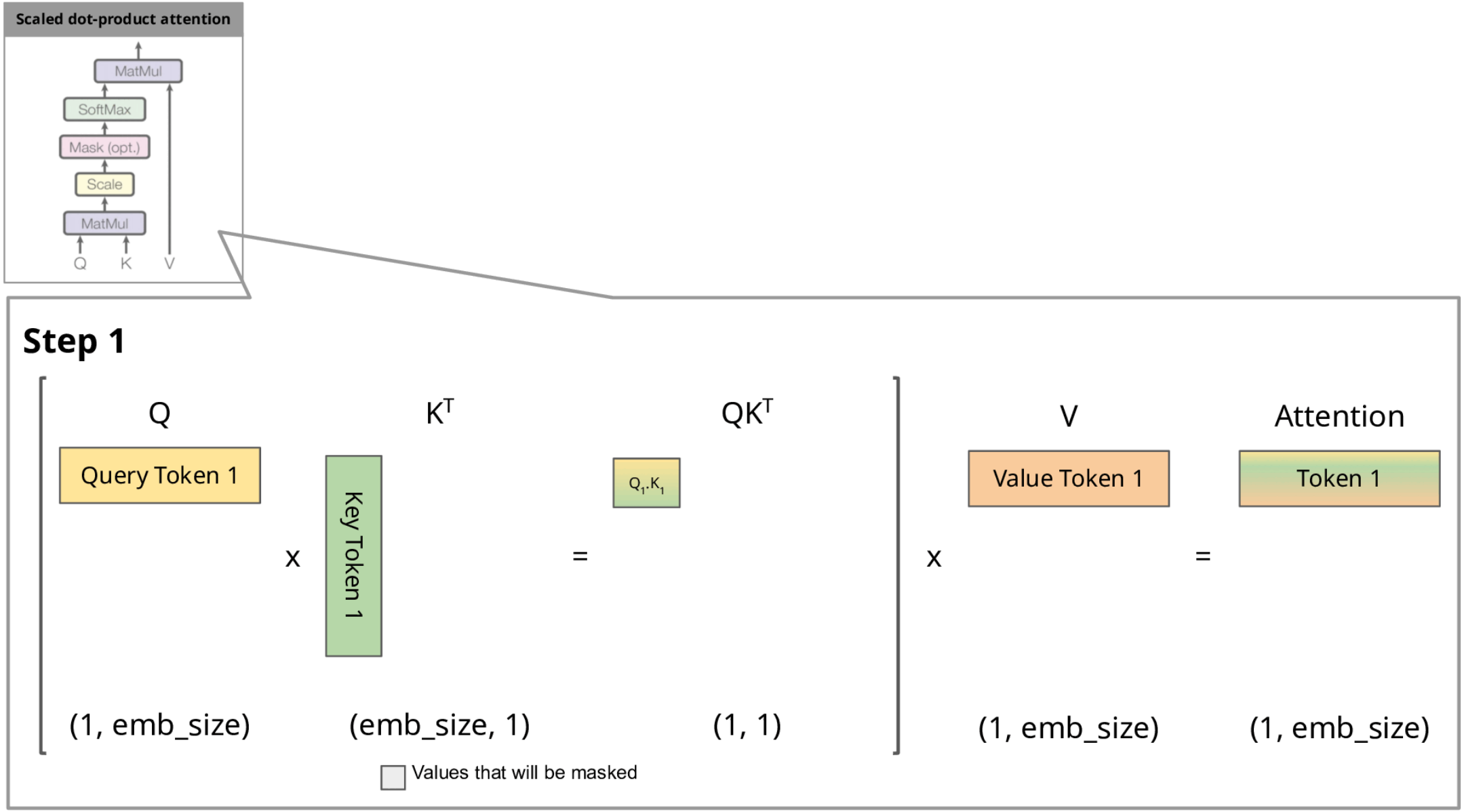
- Goal:
  - moderate size vocabulary
  - no OOV
- Idea:
  - represent rare words (OOV) by sequence of subwords
- Byte Pair Encoding (BPE)
  - not necessarily semantic meaningful
  - Originally for data compression Philip Gage. A New Algorithm for Data Compression, 1994

Online Demo:
https://koala.sh/tools/free-gpt-tokenizer
https://belladoreai.github.io/llama-tokenizer-js/example-demo/build/

KV-Cache

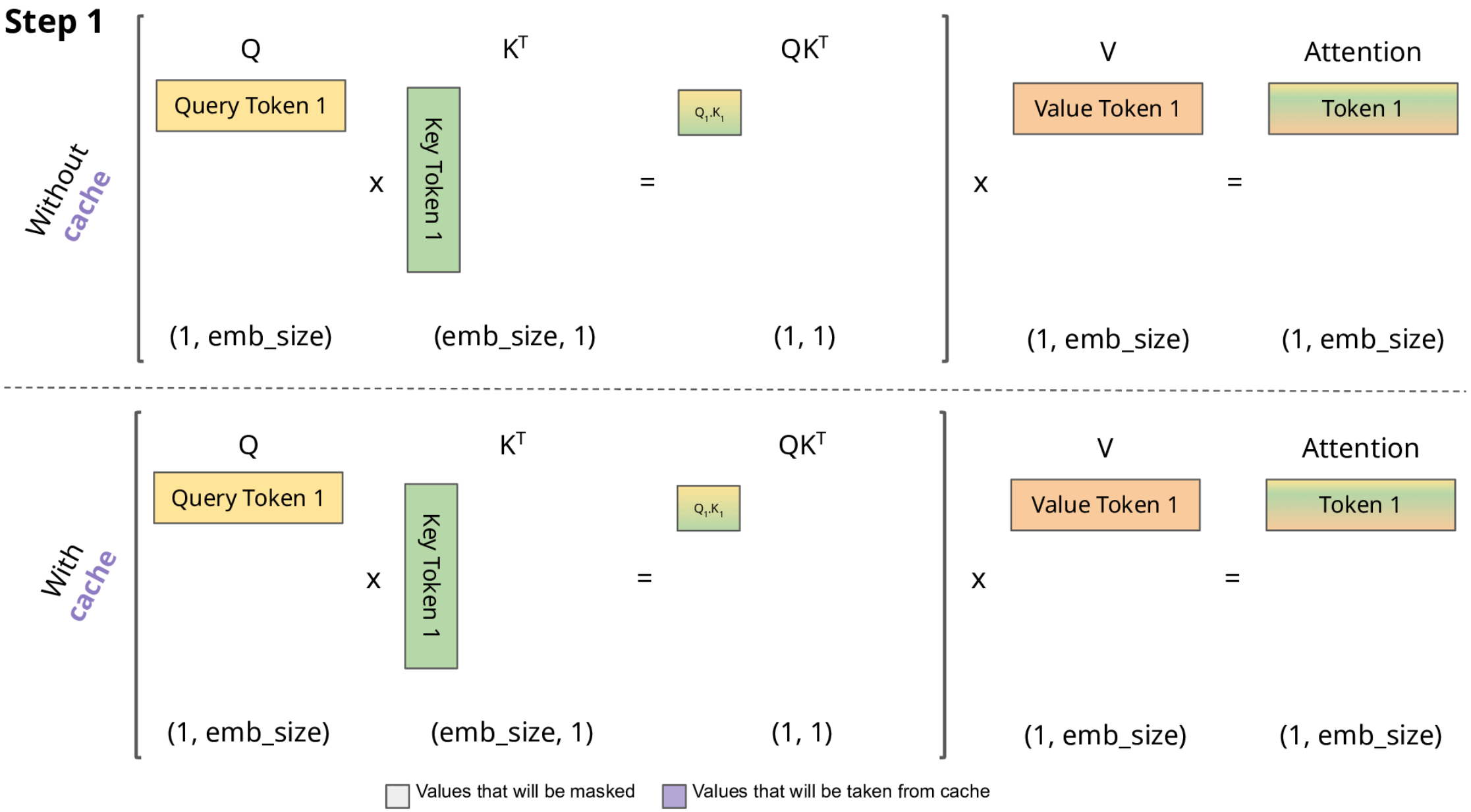


KV-Cache



Zoom-in! (simplified without Scale and Softmax)

KV-Cache


Step 1
Without cache
Q
Query Token 1
(1, emb_size)
x
K^T
Key Token 1
(emb_size, 1)
=
QK^T
Q1.K1
(1, 1)
x
V
Value Token 1
(1, emb_size)
=
Attention
Token 1
(1, emb_size)
With cache
Q
Query Token 1
(1, emb_size)
x
K^T
Key Token 1
(emb_size, 1)
=
QK^T
Q1.K1
(1, 1)
x
V
Value Token 1
(1, emb_size)
=
Attention
Token 1
(1, emb_size)
Values that will be masked
Values that will be taken from cache

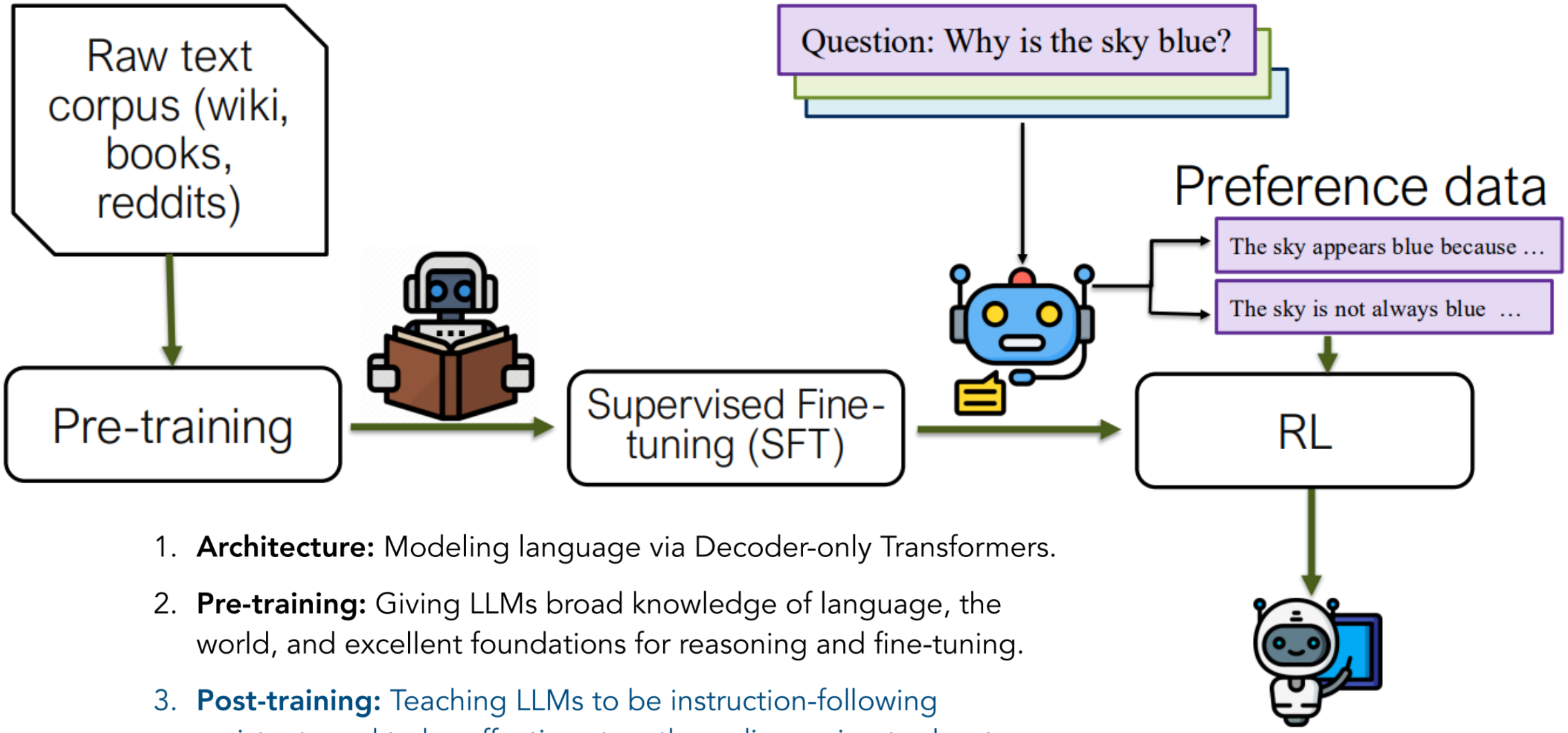
LLM Learning Targets

What might we want from foundation models?

1. **Language:** Understand our requests.
2. **Knowledge:** Know enough about the world to be useful.
3. **Retrieval and Tool Use:** Search for information if it's not sure.
4. **In-Context Learning:** Learn new skills quickly from examples.
5. **Alignment:** Actually follow our instructions.
6. **Capability:** Be effective at following them.
7. **Safety:** But don't do destructive things, even if asked.
8. **Reasoning:** Think longer, consider alternatives, and correct itself if needed.
9. …

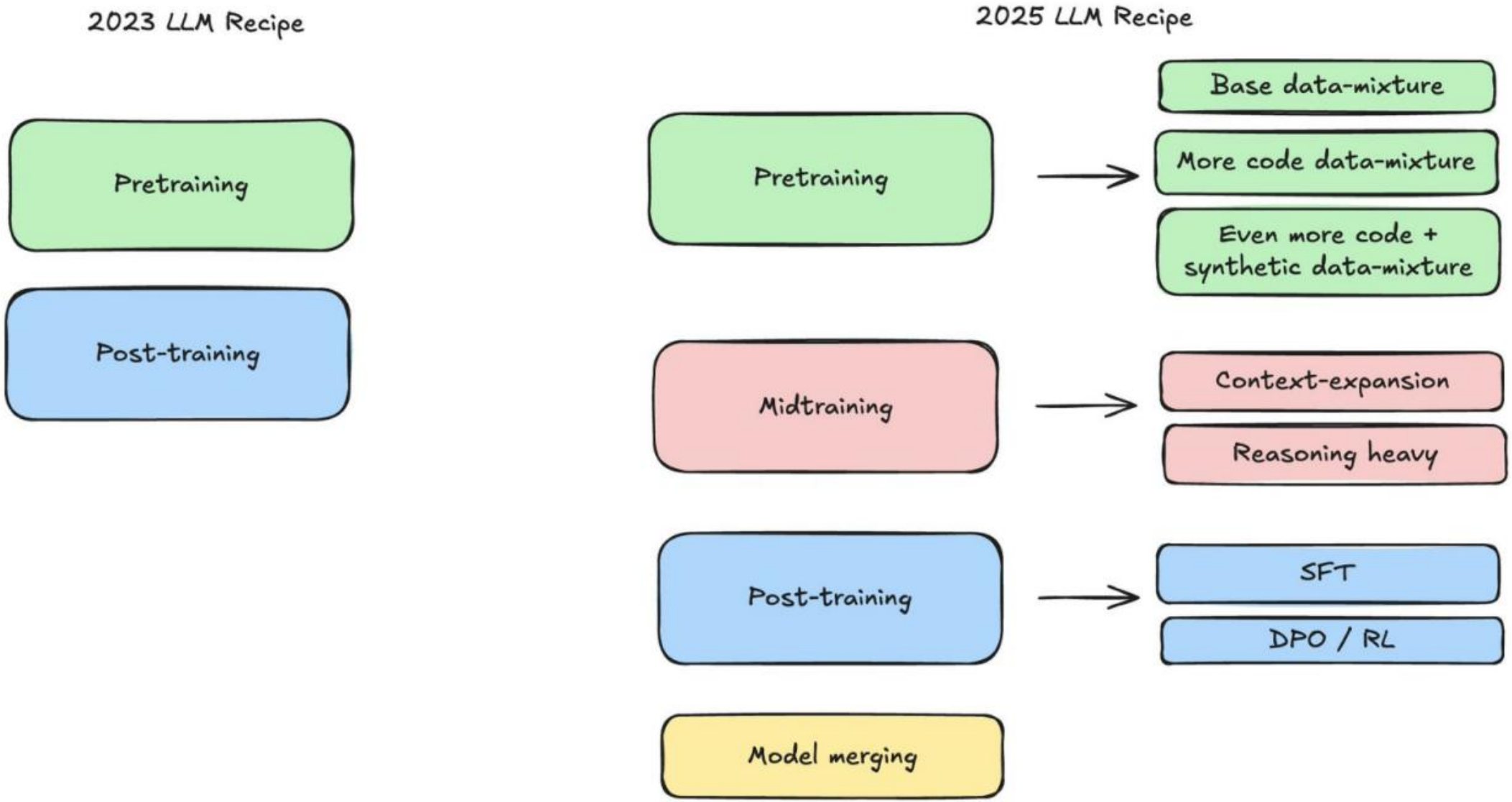
How?

LLM Learning Framework



1. **Architecture:** Modeling language via Decoder-only Transformers.
2. **Pre-training:** Giving LLMs broad knowledge of language, the world, and excellent foundations for reasoning and fine-tuning.
3. **Post-training:** Teaching LLMs to be instruction-following assistants and to be effective at math, coding, using tools, etc.

LLM Recipe


2023 LLM Recipe
Pretraining
Post-training
2025 LLM Recipe
Pretraining
Base data-mixture
More code data-mixture
Even more code + synthetic data-mixture
Midtraining
Context-expansion
Reasoning heavy
Post-training
SFT
DPO / RL
Model merging

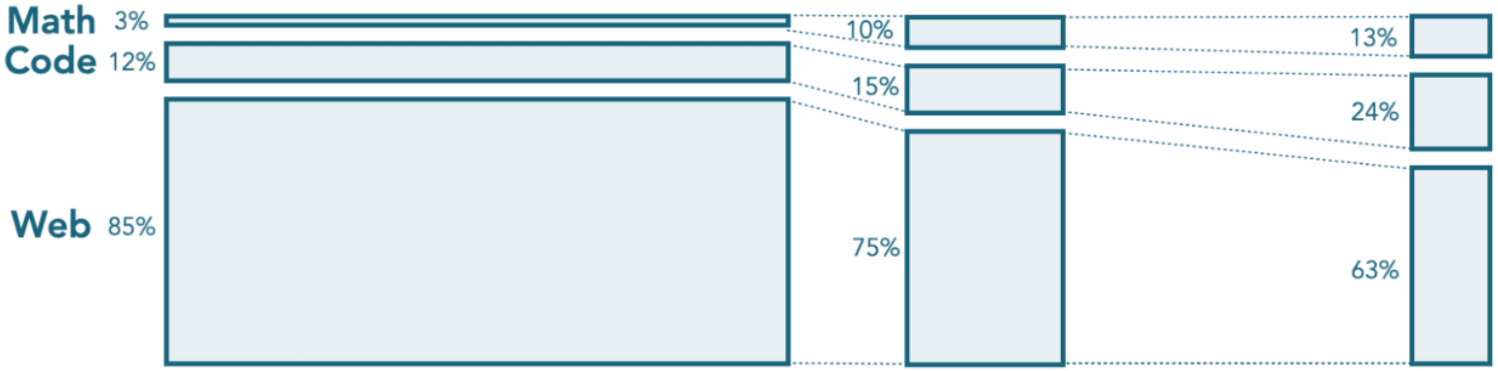
Pre-training of LMMs

- Data: raw text corpus
  - the larger the better, often crawled from the web
  - quality is important: Wikipedia, books, filtered Reddit posts
- Design of Model Architecture
  - Sparse Attention, Mixture of Expert, etc.
- Training loss:
  - cross-entropy loss for next token prediction.

Example of Pretraining Data

Example of Data Details. For SmolLM3.

Pretraining Recipe



Phase I

Description: Base training

Duration: 8T tokens

Datasets: Base mix for pretraining

Web: FineWeb-Edu, DCLM, FineWeb2, FineWeb2-HQ

Code: The Stack v2 (16 langs), StarCoder2 PRs, Jupyter/Kaggle NBs, GH issues, StackExchange

Math: FineMath3+ | InfiWebMath3+

Phase II

Description: High quality injection

Duration: 2T tokens

Datasets: Adding Stack-Edu, FineMath4+, InfiWebMath4+, MegaMath (incl. Qwen Q&A, Pro synthetic rewrites, and text code interleaved blocks)

Phase III

Description: LR Decay

Duration: 1.1T tokens

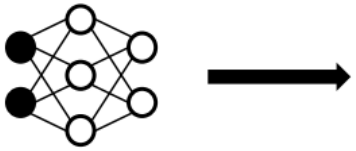
Datasets: Upsampling high quality code/math datasets and adding instruction/reasoning data such as OpenMathReasoning

LR Schedule

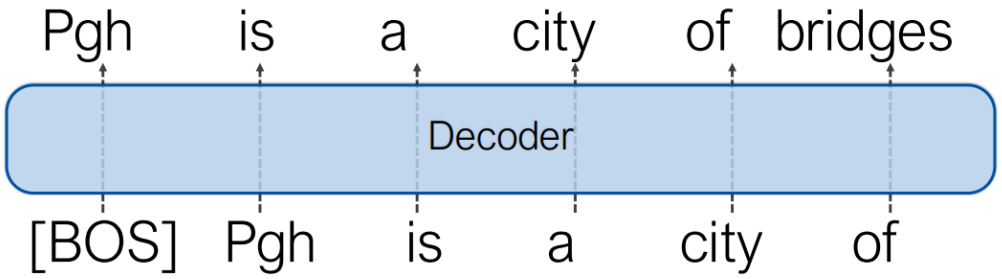
Probability Model for Next Token

P(next word y_t | Prompt x, previous words $y_{1:t-1}$)

Santa Barbara has very nice ____



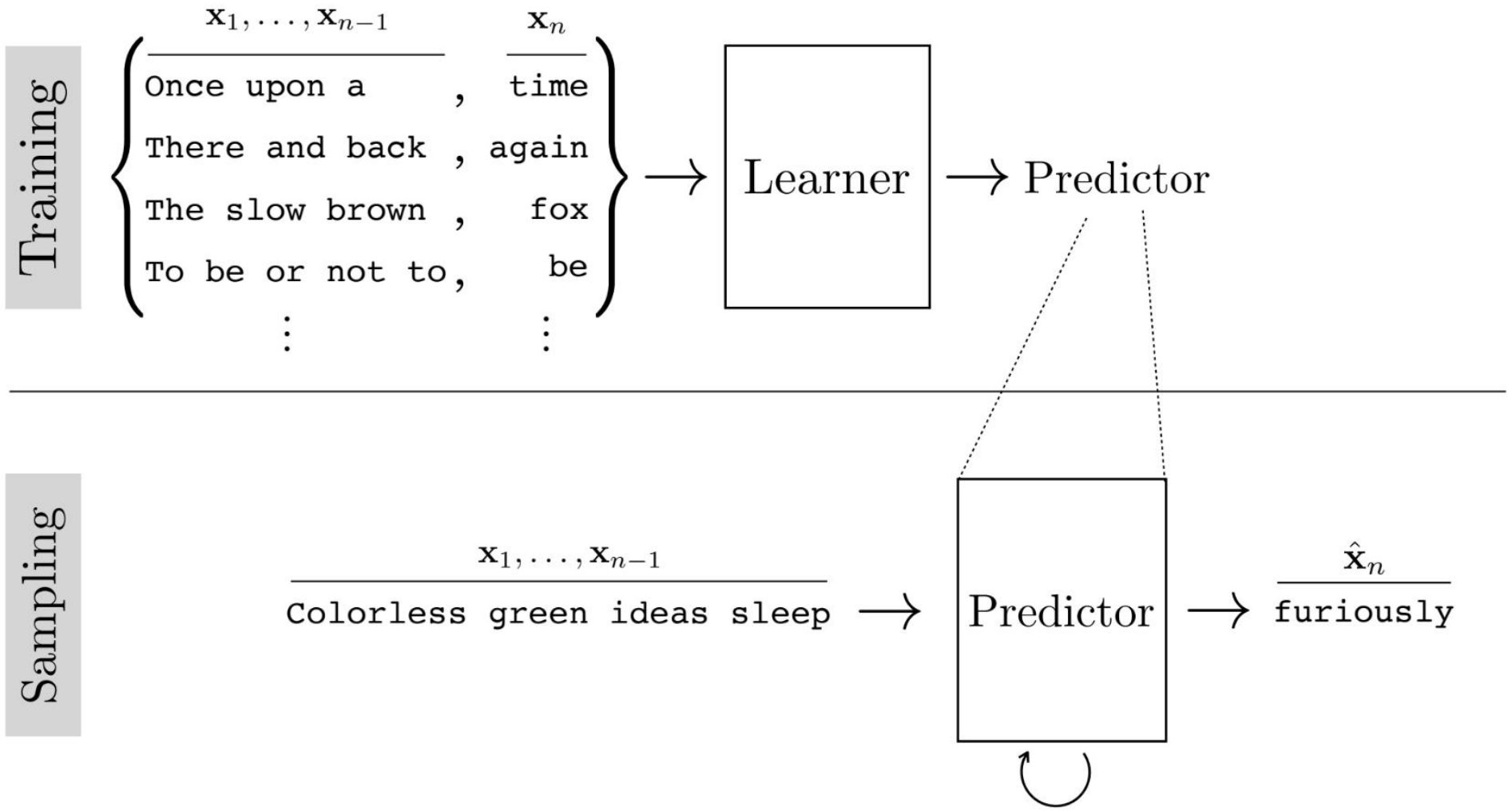
beach	0.5
weather	0.4
snow	0.01



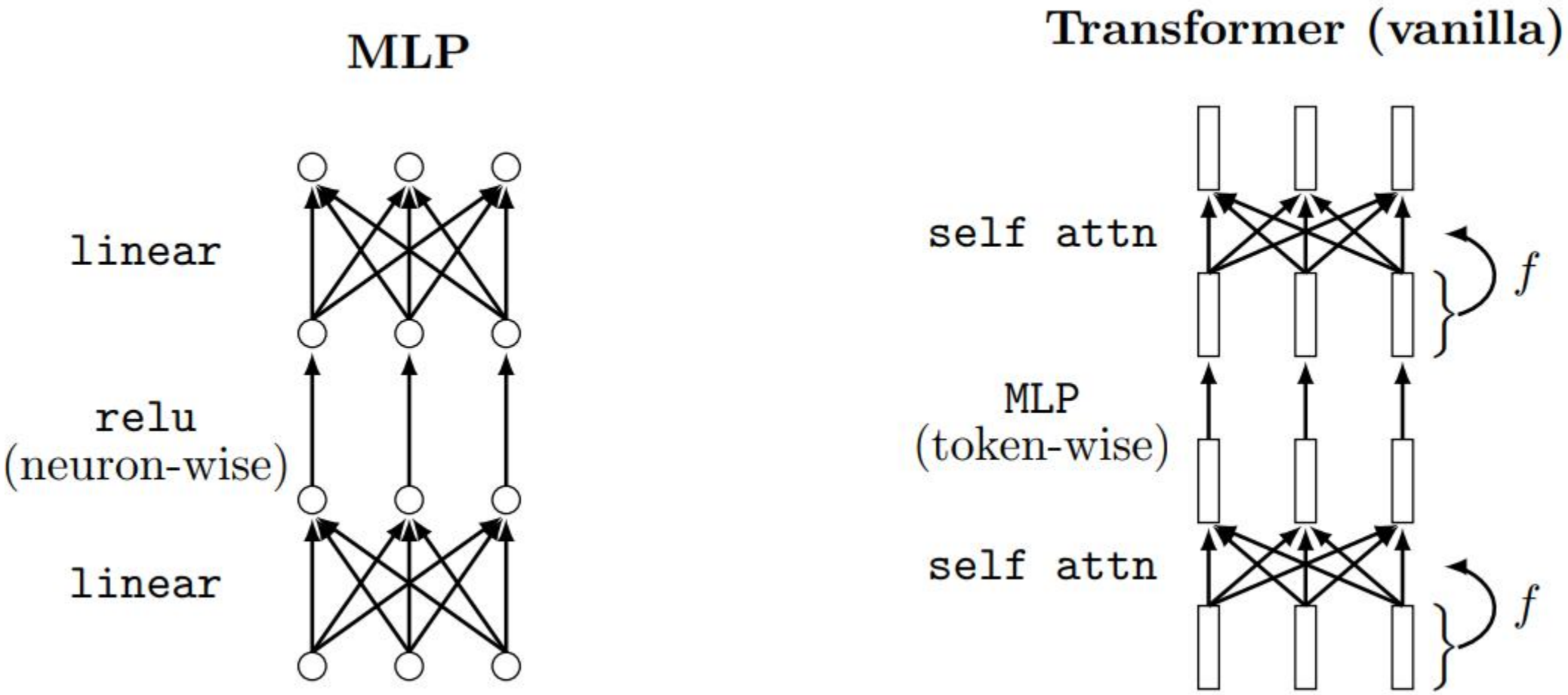
$$\text{Prob.}(x_{1..T}) = \prod_{t=1}^{T} P(x_{t+1}|x_{1..t})$$

Predicting using Neural Nets

Autoregressive Models



New Data Type: Tokens

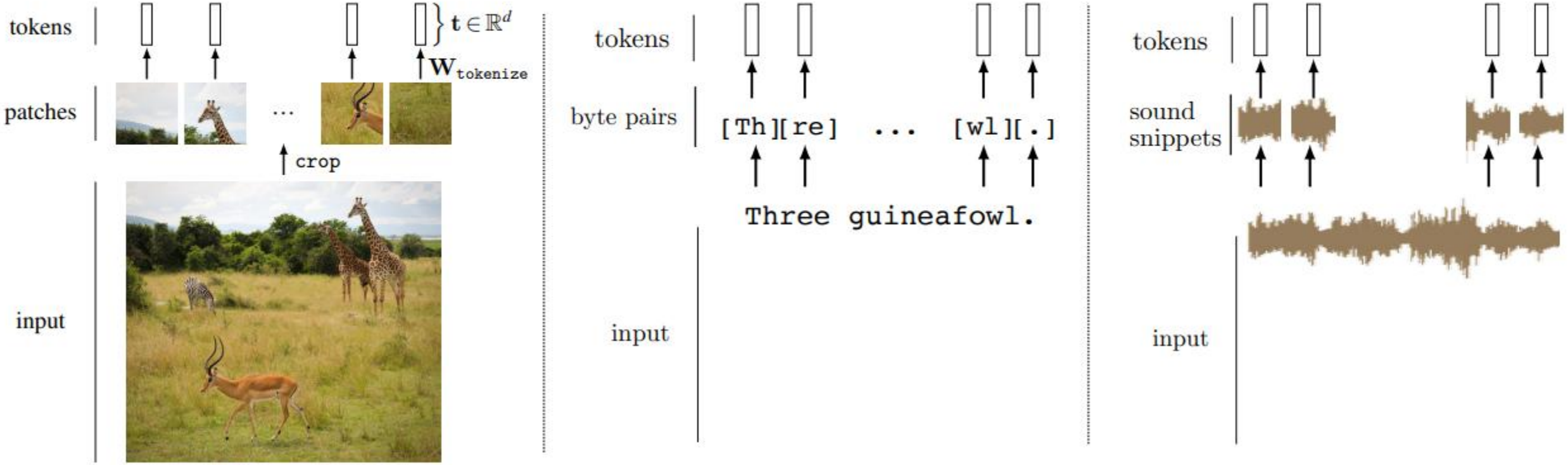


- A token is just a vector of neurons
- A token is an encapsulated bundle of information

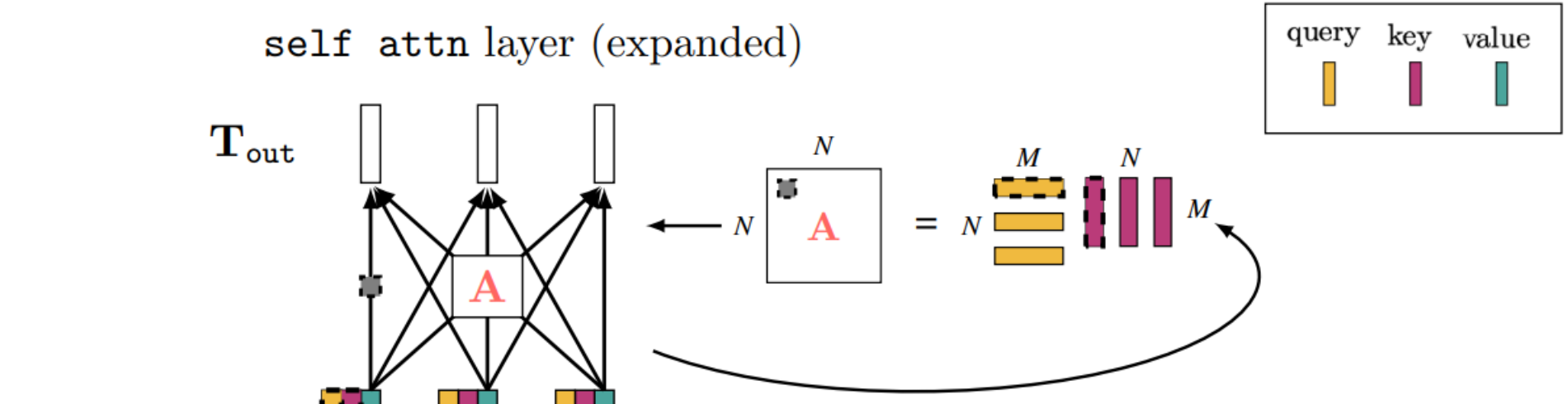
Tokenizing the Input Data

You can tokenize anything.

General strategy: chop the input up into chunks, project each chunk to a vector.



Attention



$$\mathbf{Q}_{\text{in}} = \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_q \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_q \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_q^\top \qquad \triangleleft \quad \text{query matrix}$$

$$\mathbf{K}_{\text{in}} = \begin{bmatrix} \mathbf{k}_1^\top \\ \vdots \\ \mathbf{k}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_k \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_k \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_k^\top \qquad \triangleleft \quad \text{key matrix}$$

$$\mathbf{V}_{\text{in}} = \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_v \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_v \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_v^\top \qquad \triangleleft \quad \text{value matrix}$$

$$\mathbf{A} = f(\mathbf{T}_{\text{in}}) = \texttt{softmax}\left(\frac{\mathbf{Q}_{\text{in}} \mathbf{K}_{\text{in}}^\top}{\sqrt{m}} \right) \qquad \triangleleft \quad \text{attention matrix}$$

$$\mathbf{T}_{\text{out}} = \mathbf{A} \mathbf{V}_{\text{in}}$$